



Universidad de Ingeniería y Tecnología
Sistemas Operativos

Avance Semana 5

Proyecto: Análisis y Simulación de C-Lock

C-Lock: An Energy-Efficient Hardware Synchronization Method for Multicore Embedded Systems

Integrante 1: Nombre Apellido

Integrante 2: Nombre Apellido

Integrante 3: Jhimy Jhoel Delgado Bazan

7 de septiembre de 2026

1. Avance Realizado

1.1. Lectura y comprensión del paper

Leímos el paper completo de C-Lock. Lo que más nos costó entender al principio fue cómo el hardware puede detener el reloj de un core sin que este pierda su estado — el mecanismo de *clock-gating* no es tan directo como parece desde el software. Una vez que lo vimos con más detalle, identificamos los componentes principales:

- **C-LockManager**: el componente central. Tiene un árbitro que decide quién accede al lock, un contador global de locks activos, y un pool por cada core.
- **Pool_i / Conflict Checker**: cada core tiene su propio pool con un *Conflict Checker* que detecta si hay conflicto para un lock dado y activa el clock-gating si corresponde.
- **Interfaz SW/HW**: el programador usa tres macros — `ADD_ITEM(id)`, `BEGIN_C_LOCK(id)` y `END_C_LOCK(id)` — para interactuar con el hardware.

Lo interesante es que desde el software se ve casi igual que un mutex clásico, pero por debajo el hardware hace todo el trabajo de sincronización y además apaga el reloj del core mientras espera, ahorrando energía.

1.2. Organización del repositorio

Creamos el repositorio en GitHub y lo organizamos en dos carpetas principales:

- `latex/`: contiene el informe dividido en secciones separadas, una por tema, para que cada integrante pueda trabajar su parte sin generar conflictos en git.
- `sim/`: contiene el esqueleto de la simulación en C, con headers en `include/` y fuentes en `src/`.

1.3. Esqueleto de la simulación en C

Definimos la arquitectura del simulador y escribimos el esqueleto inicial. Los módulos que tenemos son:

- `c_lock.h`: tipos y constantes compartidas (`CoreState`, modelo de energía, tamaños máximos).
- `c_lock_manager.h/.c`: gestor central — maneja la tabla de locks, las colas de espera y notifica a los cores cuando el lock queda libre.
- `core.h/.c`: representa un core del procesador; el diseño contempla usar `pthread_cond_wait` para emular el clock-gating sin busy-waiting.
- `benchmark.c`: estructura del escenario de prueba (N cores, 1 lock compartido); la lógica interna está pendiente de implementar.

El esqueleto compila sin errores. La implementación de la lógica de sincronización está en progreso y es el foco principal para la semana 6.

2. Porcentaje de Participación

Integrante	%	Actividades principales
Nombre 1	100 %	
Nombre 2	100 %	
Nombre 3	100 %	

Cuadro 1: Participación por integrante — Avance Semana 5.

3. Dificultades Encontradas

La principal dificultad fue entender el mecanismo de *clock-gating* a nivel de hardware. El paper lo describe desde el punto de vista del diseño digital, asumiendo que el lector conoce cómo funciona la distribución de reloj en un chip. Nos tomó un rato entender que detener el reloj de un core no es lo mismo que apagarlo — el estado se preserva, solo deja de ejecutar instrucciones. Revisamos algo de documentación complementaria para aclararlo.

Por el lado de la simulación, el desafío fue pensar cómo representar el clock-gating en software. No existe una forma directa de “detener” un hilo sin consumir CPU; la alternativa que encontramos es usar `pthread_cond_wait`, que bloquea el hilo sin busy-waiting. Es una aproximación, pero creemos que captura bien la idea de que el core deja de ejecutar mientras espera el lock. Queda pendiente implementarlo y validarlo.

4. Siguientes Pasos

Para la semana 6 tenemos planeado:

1. **Implementar la lógica del simulador:** completar `c_lock_manager.c` y `core.c` con la lógica real de sincronización y clock-gating, y validar que el comportamiento sea correcto antes de medir nada.
2. **Agregar escenario spin-lock:** una vez que C-Lock funcione, implementar un escenario equivalente con spin-lock clásico para poder comparar el consumo de energía entre ambos mecanismos.
3. **Arrancar con el análisis de Selfie:** necesitamos revisar el código de Selfie para entender qué partes habría que modificar para soportar C-Lock — o al menos una versión simplificada. Esto lo dejamos para la semana 6 porque primero queríamos tener claro el paper.
4. **Redactar el informe final:** con la base que tenemos en `latex/sections/`, ir completando las secciones de arquitectura y simulación a medida que avancemos con el código.

5. Uso de Herramientas de Inteligencia Artificial

Usamos dos herramientas de IA durante esta semana:

- **NotebookLM (Google)**: lo usamos para leer y procesar el paper. Nos ayudó a navegar el documento, identificar las secciones clave y hacerle preguntas directas sobre el contenido para entenderlo mejor.
- **Claude (Anthropic)**: lo usamos como guía para organizar el trabajo de la semana — estructurar el repositorio, definir cómo dividir las secciones del informe entre los integrantes, y aclarar dudas sobre el paper que no quedaban del todo claras con la lectura directa.

En ambos casos las herramientas nos ayudaron a entender y organizarnos, pero el análisis, las decisiones de diseño y la redacción del informe son nuestros.

6. Evidencias del Proceso

6.1. Estructura del repositorio

A continuación se muestra la estructura del repositorio al final de esta semana:

```
1 so_project1/  
2   latex/  
3     preamble.tex  
4     refs.bib  
5     figures/  
6     avance5/      <- Este informe  
7       main.tex  
8       sections/  
9     sections/      <- Informe final (en construccion)  
10  sim/  
11    Makefile  
12    include/  
13      c_lock.h  
14      c_lock_manager.h  
15      core.h  
16  src/  
17    c_lock_manager.c  
18    core.c  
19    benchmark.c
```

Listing 1: Estructura del repo en git.

6.2. Diagrama de arquitectura C-Lock

(Diagrama pendiente — se incluirá en el siguiente avance.)

6.3. Extracto del código — Header principal

```
1 /* c_lock.h - Tipos compartidos del simulador C-Lock */  
2 #ifndef C_LOCK_H  
3 #define C_LOCK_H  
4  
5 #define MAX_CORES      16  
6 #define MAX_LOCKS      64  
7 #define MAX_WAITERS    MAX_CORES  
8  
9 typedef enum { RUNNING, GATED } CoreState;  
10  
11 typedef struct { int owner; int waiters[MAX_WAITERS]; int nwaiters; }  
    LockEntry;  
12 typedef struct { int core_id; CoreState state; long cycles_active; long  
    cycles_gated; } Core;  
13  
14 #endif
```

Listing 2: c_lock.h — tipos y constantes del simulador.